

Guía de Implementación de Base de Datos Vectorial para RAG

Los desarrolladores deben completar secuencialmente las siguientes fases:

1. **Paso 1: Ingesta de Documentos (Document Ingestion):** El primer paso consiste en analizar y procesar los documentos de origen (ya sean manuales en PDF, archivos HTML o textos libres). Estos documentos deben dividirse en unidades de recuperación manejables mediante una estrategia de fragmentación (chunking) y, posteriormente, cada fragmento debe ser convertido a un vector utilizando un modelo de incrustación (embedding model).
2. **Paso 2: Almacenamiento e Indexación (Storage and Indexing):** Una vez generados los vectores, se deben insertar en la base de datos elegida. En esta fase es vital configurar la indexación: los equipos deben seleccionar una métrica de distancia (como la similitud del coseno) que sea compatible con su modelo de incrustación y ajustar los parámetros de búsqueda aproximada (como los algoritmos HNSW) según los requerimientos de velocidad de su sistema.
3. **Paso 3: Recuperación (Retrieval):** Finalmente, se debe implementar la lógica de búsqueda. Cuando el usuario hace una pregunta, esta se convierte en un vector y la base de datos debe recuperar y priorizar (re-ranking) los fragmentos de texto más similares semánticamente para inyectarlos al modelo de lenguaje local.

Evaluación de la Infraestructura Vectorial

El componente más crítico del sistema en esta fase es la base de datos vectorial. La decisión sobre qué motor implementar define la complejidad del despliegue y la latencia del sistema.

Análisis de Opciones Arquitectónicas Vectoriales

Los equipos deben fundamentar su selección basándose en el volumen de datos y el hardware disponible:

- **ChromaDB:** Ideal para la etapa inicial de prototipado rápido. Su filosofía de "llegar y usar" elimina la fricción de la configuración de servidores.
- **Qdrant:** Excelente opción de alto rendimiento escrita en Rust. Permite filtrado de metadatos complejo y gestiona la memoria con gran eficiencia.
- **pgvector:** La decisión arquitectónica más sólida si el equipo ya usa PostgreSQL. Permite tratar a los vectores como columnas tradicionales.
- **LanceDB:** Ofrece una arquitectura "Serverless / Embedded" (como un SQLite de vectores) que funciona directamente en disco sin procesos en segundo plano.

Criterios de Validación y Mejores Prácticas Vectoriales

Para comprobar que la base de datos se implementó adecuadamente, los evaluadores deben revisar los siguientes elementos arquitectónicos:

1. **Capa de Incrustación (Embedding Layer):** El equipo debe justificar el modelo utilizado

para transformar los textos en vectores (por ejemplo, modelos locales o especializados en español).

2. **Estrategia de Fragmentación (Chunking):** Los documentos deben segmentarse respetando fronteras semánticas (párrafos lógicos) y mantener un solapamiento (overlap) para evitar la pérdida de contexto entre los fragmentos. Inyectar textos crudos comprometerá la respuesta del LLM.

¿Qué constituye un buen resultado?

Un resultado exitoso en la implementación técnica de la base de datos vectorial no es solo que el sistema "devuelva respuestas". Un buen resultado se define por alcanzar un equilibrio medible entre precisión y velocidad. Para cargas de trabajo RAG típicas (utilizando vectores estándar de 1536 dimensiones y buscando las 10 coincidencias principales o top-K=10), **el sistema debe lograr una tasa de recuerdo (recall) del 90% al 95%, manteniendo una latencia en el percentil 95 (p95) por debajo de los 100 milisegundos.**¹ A nivel cognitivo, un buen resultado significa que las respuestas del LLM son altamente relevantes a la consulta del usuario y 100% fieles al texto de la base de datos, sin generar alucinaciones.

Evaluación del Sistema RAG (El Conocimiento Estático)

La evaluación de la recuperación de información no debe basarse en apreciaciones subjetivas. Se exige la instrumentación de métricas algorítmicas estandarizadas (como las proporcionadas por el marco RAGAS) que puntúen cuatro dimensiones críticas:

1. **Precisión del Contexto (Context Precision):** Evalúa si los fragmentos de información verdaderamente relevantes para la consulta del usuario fueron recuperados y posicionados en los primeros lugares (top-k) de la base de datos vectorial.
2. **Recuerdo del Contexto (Context Recall):** Mide si el texto recuperado por el sistema abarca toda la información necesaria para responder exhaustivamente a la pregunta del usuario.
3. **Fidelidad (Faithfulness):** Evalúa si la respuesta generada por el modelo se fundamenta estricta y únicamente en el contexto proporcionado por la base de datos. Si el modelo alucina añadiendo información externa, el puntaje debe penalizarse.
4. **Relevancia de la Respuesta (Answer Relevancy):** Califica qué tan directamente la respuesta del LLM aborda la inquietud original del usuario, penalizando las divagaciones.

Dinámica de Trabajo y Entregables (Equipos de 3 Personas)

El desarrollo de este sistema exige una división de roles estructurada, auditable a través de repositorios de control de versiones. Una carga de trabajo compartida igualmente es un criterio fundamental para el éxito y la evaluación.

Asignación de Roles Recomendada

1. **Arquitecto de Datos y Vectorización:** Responsable del pipeline de datos. Diseña los scripts de limpieza de documentos (PDFs, textos), define la estrategia de *chunking* (tamaño y solapamiento) y despliega la base de datos vectorial (ChromaDB, pgvector, etc.). Su métrica de éxito es la correcta ingesta y la persistencia de los índices.
2. **Ingeniero de Modelos LLM y Lógica Cognitiva:** Configura los servicios locales (Ollama, LMStudio), selecciona los modelos base y redacta los *System Prompts* restrictivos para el sistema RAG. Conecta la consulta del usuario con la base de datos y el modelo para generar la respuesta.
3. **Arquitecto de Pruebas (QA) y Evaluador de Sistemas:** Implementa el servidor o interfaz principal y es el encargado de instrumentar las métricas automatizadas (precisión, recuerdo, fidelidad). Configura paneles de observabilidad y lidera las pruebas de control de alucinaciones.

Requisito de Entrega: Documentación en Video

Como criterio obligatorio para la aprobación del proyecto, **cada equipo deberá grabar un video demostrativo documentando sus hallazgos**. En este video, los tres miembros deberán explicar la arquitectura elegida, mostrar el flujo de ingesta de la base de datos vectorial, evidenciar las métricas obtenidas (demostrando qué es un "buen resultado" en su caso) y hacer una prueba en vivo del modelo local contestando consultas. **Este video debe ser publicado y compartido en la red social de preferencia del equipo**, fomentando así el desarrollo de su portafolio profesional público y la divulgación técnica.

Rúbrica General de Evaluación Arquitectónica (Fase Vectorial)

La siguiente rúbrica establece el estándar general para evaluar la adopción de la base de datos vectorial y el flujo RAG de manera independiente a la temática del proyecto.

Categoría Evaluada	4 - Sobresaliente (Arquitectura Robusta)	3 - Competente (Prototipo Sólido)	2 - En Desarrollo (Funcional pero Frágil)	1 - Insuficiente (Fallo Crítico)
Arquitectura de Base de Datos Vectorial	Selección arquitectónica justificada. Cumple la meta de buen resultado: recall > 90% y latencias submilisegundo (<100ms).	Base de datos persistente ejecutando búsquedas puramente vectoriales. Recuperación precisa en la gran mayoría de las	Configuración de BD volátil (los datos se borran al reiniciar). Latencias inestables. Recuperación con alto nivel de ruido semántico	Falla la librería de vectorización o despliegue. La DB colapsa bajo cargas mínimas o devuelve constantemente errores y/o resultados vacíos.

	Indexación optimizada.	consultas, aunque con latencias levemente mayores.	(documentos irrelevantes).	
Pipeline de Datos y Fragmentación (Chunking)	Limpieza de datos impecable. Fragmentación semántica inteligente con superposición (overlap) calculada y justificada. No se pierde el contexto entre fragmentos.	Fragmentación por tamaño estático (ej. 500 caracteres fijos) con cierto solapamiento. Preserva el contexto general aunque rompe algunas oraciones.	Fragmentación rudimentaria sin limpieza previa de datos (dark/dirty data). Pérdida de contexto constante que afecta directamente a los embeddings.	Ingesta de documentos completos sin fragmentar. El sistema excede sistemáticamente el límite de tokens de los modelos de <i>embedding</i> o del LLM.
Métricas RAG y Contención de Alucinaciones	Implementación de métricas de evaluación (Fidelidad, Relevancia, Precisión) con resultados excelentes. El <i>System Prompt</i> restringe al LLM para que admita ignorancia si el dato no está en la DB Vectorial.	Prompting estricto que restringe adecuadamente al modelo. Respuestas correctas y fundamentadas en el contexto en su mayoría. Revisiones visuales consistentes sin alucinaciones obvias.	Ausencia de métricas de validación. El modelo frecuentemente ignora el contexto de la base de datos para responder usando su conocimiento pre-entrenado.	El sistema genera alucinaciones categóricas y responde con información inventada. No existe contención entre el texto recuperado y la respuesta del LLM.
Trabajo en Equipo y Entregables (Video)	Trabajo equitativo (3 roles claros). Video publicado en redes sociales explicando métricas, arquitectura y demostrando el sistema en vivo con excelencia técnica.	Roles distribuidos con código funcional. El video fue publicado y cubre la demostración, pero carece de profundidad al explicar los hallazgos técnicos.	Carga de trabajo desigual. El video es rudimentario, no participan todos los integrantes o no logran explicar la arquitectura de la base de datos vectorial.	Trabajo colaborativo nulo. Software inoperante y/o el equipo no entregó ni publicó el video explicativo en sus redes sociales.